

Enterprise AI Architect Learning Path v3 – Phase Guide 1

Enterprise AI Architect Learning Path v3 – Phase Guide 1

Python & AI Engineering Foundations

Purpose of This Phase

This phase exists to establish the technical foundation that will support every subsequent project in the learning path.

Although the long-term goal is becoming an Enterprise Agentic AI Architect, the immediate objective is much simpler:

Learn to build reliable AI-adjacent software in Python using the same engineering discipline that has been applied throughout a professional .NET career.

Many AI practitioners begin with prompts and notebooks. Enterprise AI architects begin with:

- Reproducible environments
- Source control
- Logging
- Testing
- Data pipelines
- Strong typing
- Architecture decisions
- Documentation

The focus of this phase is not machine learning. It is software engineering for AI systems. The skills developed here will be used in every later phase, including Retrieval-Augmented Generation (RAG), Agents, MCP Servers, and LLMOps.

Phase Objectives

Upon completion of this phase, you should be able to:

- Build Python applications using modern development practices
- Create maintainable project structures
- Work comfortably with Python OOP
- Use AsyncIO for concurrent operations
- Create data processing pipelines
- Build command-line utilities
- Implement structured logging
- Write unit tests with Pytest
- Manage configuration and secrets

- Produce architecture documentation and ADRs

Architect Mindset

As a senior .NET developer, your goal is **not** to become a Python expert.

Your goal is to become:

|

An architect who can leverage Python effectively within AI ecosystems.

Think of Python as:

	☰ .NET Experience	☰ Python Equivalent
1	C# Classes	Python Classes / Dataclasses / Pydantic Models
2	Task / Async-Await	AsyncIO
3	Dependency Injection	Constructor Injection / Composition
4	NuGet	uv / pip
5	appsettings.json	.env + Configuration Models
6	xUnit / MSTest	Pytest
7	ILogger	Structured Logging
8	ASP.NET APIs	FastAPI

Many concepts are familiar. The syntax is different; the engineering principles are not.

Primary Resources

Required

R0

100 Days of Code: Python Pro Bootcamp

Focus Areas:

- OOP
- File I/O
- APIs
- Error Handling

- Async Programming
- Testing

R1

AI Engineering (Chip Huyen)

Focus Areas:

- AI application architecture
 - Data pipelines
 - Evaluation thinking
 - AI engineering mindset
-

Core Technical Skills

1. Python Fundamentals

Master:

- Functions
- Classes
- Modules
- Packages
- Virtual environments
- Error handling

Do not obsess over mastering every Python feature.

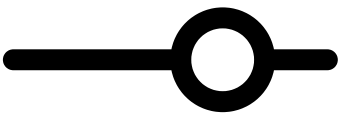
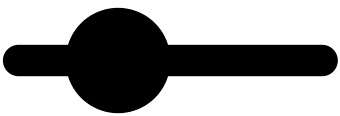
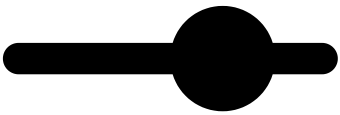
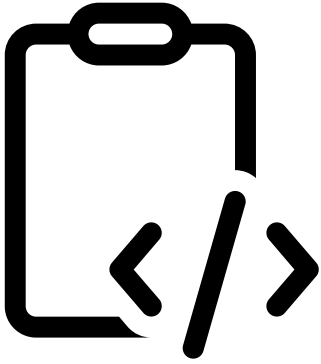
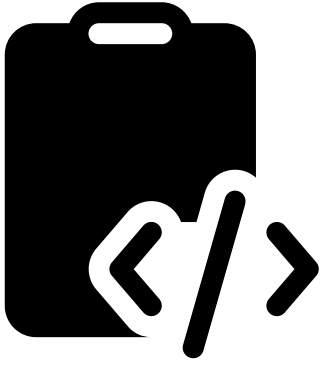
Focus on becoming comfortable enough to build production-quality tooling.

2. Object-Oriented Design

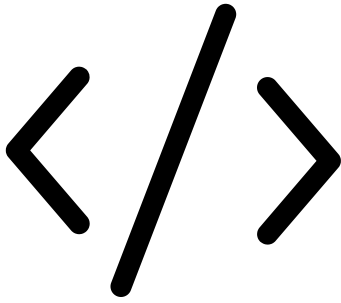
Create and work with models such as:

≡

≡



▼



Python

1

```
class Document:
```

2

```
    pass
```

3

4

```
class Chunk:
```

5

```
    pass
```

6

7

```
class SourceMetadata:
```

8

```
    pass
```

9

Show more lines

These models become foundational components in later RAG systems.

3. File Processing

Learn to:

- Read files
- Write files
- Parse JSON
- Parse Markdown
- Traverse directories
- Validate file content

Future project phases depend heavily on processing content from multiple sources.

4. Configuration Management

Learn:

- Environment variables
- .env files
- Configuration objects
- Secret handling

Avoid hard-coding:

- API Keys
 - Connection strings
 - Model names
 - Environment-specific settings
-

5. Structured Logging

Every production system needs visibility.

Learn:

- Log levels
- Structured logs
- Error logging
- Correlation identifiers

Goal:

Be capable of troubleshooting a failing ingestion pipeline from logs alone.

6. Testing

Implement:

- Unit tests
- Validation tests
- Happy-path tests

- Failure tests

Suggested standard:

- Every significant module contains tests.
 - Every bug receives a regression test.
-

7. Async Programming

Focus on:

- async
- await
- asyncio

This becomes critical later when:

- Calling APIs
 - Processing large numbers of documents
 - Running agent workflows
 - Executing multiple tools simultaneously
-

Phase Project

RPG Catalog Ingestor

This is the first portfolio project component.

Purpose

Create a system capable of scanning a directory of RPG resources and converting them into a normalized internal representation.

Future phases will use this output as the foundation for retrieval systems.

Functional Requirements

The system should:

Scan Content

Support:

- Markdown
- Text
- JSON

Future formats may include:

- PDF
 - Word Documents
-

Extract Metadata

Examples:

- Title
 - Source File
 - Category
 - System
 - Author
 - Tags
-

Normalize Content

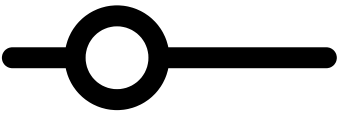
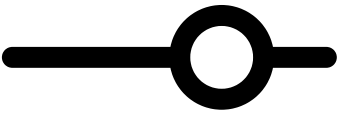
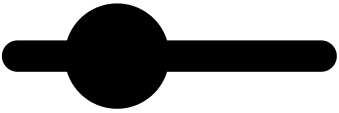
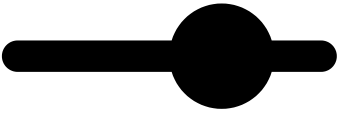
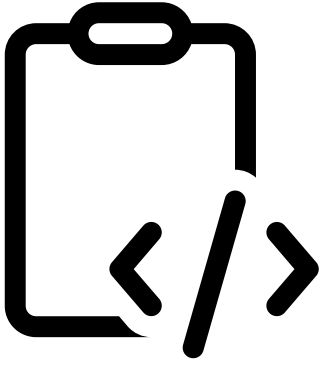
Transform multiple formats into a common structure.

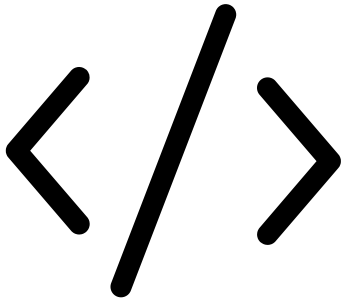
Example:

|||

|||







JSON

```
1
{
2
  "id": "",
3
  "title": "",
4
  "content": "",
5
  "metadata": {}
6
}
7
``
8

```

Show more lines

Generate Output

Produce structured JSON suitable for:

- Embedding generation
- Chunking
- Search indexing

in later phases.

Architecture Deliverables

Architecture Decision Record (ADR)

Create at least three ADRs:

ADR-001

Repository Structure

Why was this structure selected?

ADR-002

Configuration Strategy

How are settings and secrets managed?

ADR-003

Document Model Design

Why were the chosen schema structures selected?

Portfolio Artifacts

Create:

Repository

Source-controlled project repository.

README

Project introduction and setup guide.

Architecture Notes

Document:

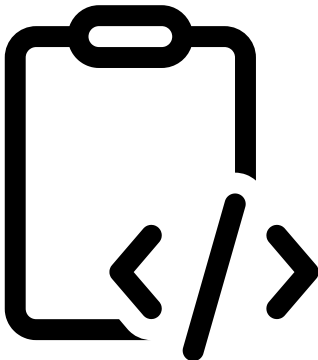
- Major decisions
- Assumptions
- Risks

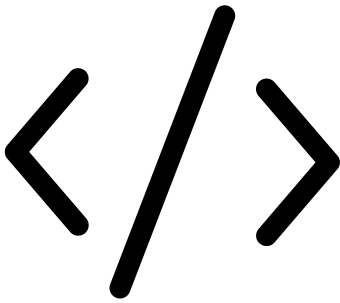
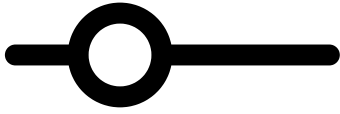
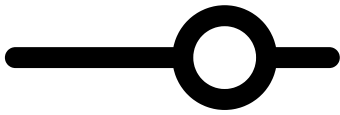
Initial Diagram

Simple high-level diagram showing:

10

10





Plain Text

1

Files

2

↓

3

Parser

4

↓

5

Normalizer

6

↓

7

JSON Output



Show more lines

Suggested Sprint Progression

Sprint 1

Environment Setup

Goals:

- Repository creation
- Git setup
- Virtual environment
- Project structure

Done When:

- Repository operational
 - Initial commit completed
-

Sprint 2

Document Models

Goals:

- Document
- Chunk
- Metadata models

Done When:

- Models created
 - Tests passing
-

Sprint 3

File Processing

Goals:

- File discovery
- Parsing
- Validation

Done When:

- Sample files processed successfully

Sprint 4

Ingestion Pipeline

Goals:

- End-to-end workflow

Done When:

- Structured JSON generated
-

Sprint 5

Testing & Hardening

Goals:

- Improve reliability
- Improve logging
- Documentation

Done When:

- Tests stable
 - Documentation complete
-

Common Pitfalls

Pitfall 1

Trying to learn all of Python first.

Avoid:

"I'll start building once I finish the Python course."

Instead:

Build while learning.

Pitfall 2

Skipping tests.

Future agent systems become significantly harder to debug.

Testing habits established here will pay dividends later.

Pitfall 3

Overengineering.

The purpose is not creating a perfect ingestion platform.

The purpose is creating a learning platform for future RAG systems.

Pitfall 4

Treating Python like C#.

Many architectural concepts transfer.

Not every C# design pattern needs a direct Python equivalent.

Definition of Done

Phase 1 is complete when:

- Python environment is established
 - Repository structure is stable
 - Ingestion pipeline functions end-to-end
 - Structured logging exists
 - Unit tests exist and pass
 - ADRs are documented
 - README is complete
 - Sample content can be transformed into normalized JSON
 - Portfolio artifacts are committed to source control
-

End-of-Phase Self-Assessment

You are ready for Phase 2 when you can confidently answer:

- Can I build and test Python applications independently?
- Can I process and normalize document content?
- Can I create maintainable project structures?
- Can I troubleshoot problems through logging and tests?
- Can I explain my architectural decisions?
- Can I extend the ingestion pipeline without major redesign?

If all answers are "yes," proceed to **Phase 2 – Azure AI Foundry & RAG Foundations**.